



**INSTITUTO POLITÉCNICO
UNIVERSIDADE KIMPA VITA**

RELATÓRIO TÉCNICO DE LÓGICA DE PROGRAMAÇÃO

TEMA: SISTEMA DE GESTÃO DE DADOS DE ESTUDANTES

Curso: 1º Ano de Engenharia Informática

Turma: TL 101

1º Ano

Período: manhã

O Docente

Moyo Kanivengidio

Uíge, Julho/ 2026

Índice

Índice.....	2
1. Introdução e Objetivo do Trabalho	1
1.1. Objetivos Gerais	1
1.2. Objetivos Específicos	1
2. Análise dos Requisitos Funcionais e Não Funcionais	2
2.1. Requisitos Funcionais (RF)	2
2.2. Requisitos Não Funcionais (RNF)	4
3. Descrição Detalhada dos Algoritmos de Ordenação e Pesquisa.....	4
3.1. Algoritmos de Ordenação.....	4
3.1.1. Bubble Sort — Ordenação por Matrícula (crescente)	4
3.1.2. Selection Sort — Ordenação por Nome (alfabética)	5
3.1.3. Insertion Sort — Ordenação por Nota (decrescente).....	6
3.2. Algoritmos de Pesquisa.....	7
3.2.1. Pesquisa Linear — Pesquisa por Nome.....	7
3.2.2. Pesquisa Binária — Pesquisa por Matrícula.....	8
3.3. Quadro-Resumo Comparativo.....	8
4. Manual de Utilização do Programa.....	10
4.1. Requisitos do Sistema	10
4.2. Compilação do Programa	10
4.3. Execução do Programa.....	10
4.4. Preparação dos Ficheiros de Dados.....	11
4.5. Navegação no Menu Principal	11
4.5.1. Submenu de Ordenação (Opção 3).....	11
4.5.2. Submenu de Pesquisa (Opção 4)	12
4.5.3. Submenu de Gravação (Opção 5).....	12
4.6. Adicionar um Novo Estudante — Passo a Passo	12
4.7. Dicas e Avisos Importantes.....	13
5. Conclusão: Aprendizados e Desafios Enfrentados	14
5.1. Principais Aprendizados.....	14
5.2. Principais Desafios Enfrentados.....	14

5.3. Possíveis Melhorias Futuras..... 15

5.4. Considerações Finais..... 15

1. Introdução e Objetivo do Trabalho

Este relatório documenta o desenvolvimento de uma aplicação de consola em linguagem C, designada “Sistema de Controlo de Alunos- GEST.UNIKIV”, cujo propósito é gerir registos de estudantes (matrícula, nome e nota), permitindo carregar dados a partir de ficheiros de texto, adicionar novos registos em memória, ordenar e pesquisar a lista de estudantes através de diferentes algoritmos, e exportar os resultados para ficheiros de texto ou binários.

O trabalho foi concebido como exercício prático de consolidação de conhecimentos fundamentais de programação em C, nomeadamente: alocação dinâmica de memória, manipulação de ficheiros, estruturas de dados (struct), validação de entradas do utilizador, e implementação manual de algoritmos clássicos de ordenação e pesquisa.

1.1. Objetivos Gerais

Desenvolver uma aplicação funcional e modular que permita gerir um conjunto de registos de estudantes de forma persistente e interativa, demonstrando o domínio dos conceitos fundamentais da linguagem C aplicados a um problema realista de gestão de dados.

1.2. Objetivos Específicos

- Implementar uma estrutura de dados (struct Estudante) e geri-la através de um array dinâmico (alocado com malloc/realloc/free).
- Implementar a leitura de dados a partir de ficheiros de texto, com seleção interativa do ficheiro a carregar.
- Permitir a inserção de novos estudantes em memória, com validação rigorosa dos dados introduzidos (matrícula, nome e nota).
- Implementar e comparar três algoritmos de ordenação distintos (Bubble Sort, Selection Sort e Insertion Sort), aplicados a critérios diferentes (matrícula, nome e nota).
- Implementar dois algoritmos de pesquisa distintos (pesquisa linear e pesquisa binária), aplicados a critérios diferentes (nome e matrícula).
- Permitir a exportação dos dados processados para ficheiros de texto estruturado e para ficheiros binários.

- Construir uma interface de utilizador baseada em menus textuais, robusta a entradas inválidas

2. Análise dos Requisitos Funcionais e Não Funcionais

Esta secção apresenta os requisitos identificados a partir da implementação do programa, organizados em requisitos funcionais (o que o sistema faz) e requisitos não funcionais (como o sistema o faz, e sob que restrições).

2.1. Requisitos Funcionais (RF)

Código	Requisito Funcional	Função(ões) Responsável(eis)
RF01	O sistema deve listar os ficheiros .txt existentes na pasta e permitir ao utilizador escolher qual carregar.	lerDadosFicheiro
RF02	O sistema deve ler registos no formato “Matrícula;Nome;Nota” e armazená-los em memória dinâmica.	lerDadosFicheiro
RF03	O sistema deve permitir adicionar um novo estudante em memória, validando matrícula, nome e nota.	adicionarEstudante, nomeValido, matriculaExiste, gerarMatriculaAleatoria
RF04	O sistema deve gerar automaticamente uma matrícula válida e única quando a inserida for inválida ou duplicada.	gerarMatriculaAleatoria, matriculaExiste
RF05	O sistema deve ordenar a lista de estudantes por matrícula (crescente), por nome (alfabética) e por nota (decrecente).	bubbleSortMatricula, selectionSortNome, insertionSortNota
RF06	O sistema deve permitir pesquisar estudantes por nome (parcial, insensível a maiúsculas/minúsculas).	pesquisaLinearNome, paraMinusculas
RF07	O sistema deve permitir pesquisar um estudante por matrícula utilizando pesquisa binária.	pesquisaBinariaMatricula
RF08	O sistema deve exibir a lista de estudantes em formato tabular alinhado.	exibirEstudantes
RF09	O sistema deve exportar os dados em memória para um novo ficheiro de texto (Matrícula;Nome;Nota).	menuGravarFicheiro (opção 1)
RF10	O sistema deve exportar os dados em memória para um ficheiro binário (.dat).	menuGravarFicheiro (opção 2)
RF11	O sistema deve impedir o acesso às operações de ordenação, pesquisa e gravação caso não existam dados carregados em memória.	main
RF12	O sistema deve libertar toda a memória alocada dinamicamente ao terminar.	main

2.2. Requisitos Não Funcionais (RNF)

Código	Requisito Não Funcional	Observação
RNF01	Usabilidade: interface de menus numerados, em português, com mensagens de sucesso, aviso e erro claramente identificadas.	Prefixos [SUCESSO], [AVISO], [ERRO]
RNF02	Portabilidade limitada: o sistema depende de comandos do sistema operativo Windows (cls, dir).	Uso de system("cls") e system("dir /B *.txt > ...")
RNF03	Gestão eficiente de memória: o array de estudantes é redimensionado dinamicamente, sem limite fixo de registos.	realloc / free
RNF04	Robustez de entrada: todas as leituras com scanf são validadas, evitando ciclos infinitos com entradas não numéricas.	Verificação do valor de retorno de scanf
RNF05	Desempenho adequado à escala esperada: os algoritmos $O(n^2)$ são aceitáveis para o volume de dados típico de uma turma (dezenas a centenas de registos).	Bubble, Selection e Insertion Sort
RNF06	Persistência de dados: os dados podem ser gravados em formato legível (texto) e em formato compacto (binário).	_Saida.txt / _Saida.dat
RNF07	Manutenibilidade: o código está organizado em funções com responsabilidades únicas e bem delimitadas.	Modularização por função

3. Descrição Detalhada dos Algoritmos de Ordenação e Pesquisa

Nesta secção descreve-se o funcionamento de cada algoritmo implementado, o seu papel concreto no programa, e a justificativa pedagógica e técnica para a escolha de cada um.

3.1. Algoritmos de Ordenação

3.1.1. Bubble Sort — Ordenação por Matrícula (crescente)

Implementado na função `bubbleSortMatricula`, este algoritmo percorre repetidamente o array, comparando pares de elementos adjacentes e trocando-os sempre que estão fora de ordem. Em cada passagem completa, o maior valor ainda não ordenado é “empurrado” para a sua posição final, à semelhança de uma bolha a subir à superfície.

```

for (i = 0; i < n - 1; i++) {

    for (j = 0; j < n - i - 1; j++) {

        if (v[j].matricula > v[j+1].matricula) {

            // troca v[j] com v[j+1]

        }

    }

}

```

Complexidade: $O(n^2)$ no caso médio e pior caso; $O(n)$ no melhor caso (com otimização de paragem antecipada, não presente nesta versão).

Justificativa da escolha: o Bubble Sort foi escolhido por ser o algoritmo mais simples de implementar e compreender, sendo didaticamente adequado para introduzir o conceito de ordenação por comparação e troca. Adicionalmente, esta função é reutilizada como pré-requisito da pesquisa binária por matrícula (secção 3.2.2), pelo que ter uma implementação simples e fiável é suficiente, já que o volume de dados (registos de estudantes de uma turma) é tipicamente pequeno, tornando o custo $O(n^2)$ negligenciável na prática.

3.1.2. Selection Sort — Ordenação por Nome (alfabética)

Implementado na função `selectionSortNome`, este algoritmo divide o array numa parte já ordenada e numa parte não ordenada. Em cada iteração, percorre a parte não ordenada à procura do menor elemento (segundo a comparação alfabética de `strcmp`) e troca-o com a primeira posição não ordenada.

```

for (i = 0; i < n - 1; i++) {

    indiceMinimo = i;

```

```

for (j = i+1; j < n; j++) {

    if (strcmp(v[j].nome, v[indiceMinimo].nome) < 0)

        indiceMinimo = j;

}

// troca v[i] com v[indiceMinimo]

}

```

Complexidade: $O(n^2)$ em todos os casos (melhor, médio e pior), pois percorre sempre toda a sublista restante para encontrar o mínimo.

Justificativa da escolha: o Selection Sort foi escolhido para a ordenação por nome porque realiza, no máximo, $n-1$ trocas de elementos, o que é vantajoso quando o custo de mover registos é considerado (cada troca envolve copiar toda a struct Estudante, incluindo o campo nome de 100 bytes). Para listas pequenas, esta característica torna o Selection Sort uma alternativa eficiente em termos de operações de cópia, além de ser um algoritmo distinto do Bubble Sort, permitindo demonstrar uma estratégia diferente de ordenação (seleção do extremo, em vez de trocas locais sucessivas).

3.1.3. Insertion Sort — Ordenação por Nota (decrescente)

Implementado na função `insertionSortNota`, este algoritmo constrói a lista ordenada um elemento de cada vez: para cada novo elemento (chave), este é comparado com os elementos já ordenados à sua esquerda e deslocado para a posição correta, de modo a manter a ordem decrescente das notas.

```

for (i = 1; i < n; i++) {

    chave = v[i];

```

```
j = i - 1;

while (j >= 0 && v[j].nota < chave.nota) {

    v[j+1] = v[j];

    j = j - 1;

}

v[j+1] = chave;

}
```

Complexidade: $O(n^2)$ no caso médio e pior caso; $O(n)$ no melhor caso (lista já ordenada na ordem desejada).

Justificativa da escolha: o Insertion Sort foi escolhido para a ordenação por nota por ser eficiente em listas pequenas ou parcialmente ordenadas (que é o caso típico de uma lista de notas inserida progressivamente pelo utilizador), e por permitir facilmente inverter o critério de comparação (< para >) para obter ordem decrescente, o que demonstra a flexibilidade do algoritmo. A escolha de três algoritmos diferentes — Bubble, Selection e Insertion — para três critérios diferentes também tem um valor pedagógico claro: permite comparar, na prática, três abordagens clássicas de ordenação por comparação dentro da mesma aplicação.

3.2. Algoritmos de Pesquisa

3.2.1. Pesquisa Linear — Pesquisa por Nome

Implementada na função `pesquisaLinearNome`, este algoritmo percorre sequencialmente todos os registos do array, convertendo tanto o termo de pesquisa como o nome de cada estudante para minúsculas (através da função auxiliar `paraMinusculas`), e utiliza a função `strstr` para verificar se o termo pesquisado ocorre como substring do nome. Todos os registos correspondentes são apresentados, juntamente com o total de ocorrências encontradas.

Complexidade: $O(n)$, pois é necessário comparar o termo de pesquisa com o nome de cada um dos n estudantes, no pior caso.

Justificativa da escolha: a pesquisa linear foi escolhida para a pesquisa por nome porque os nomes não estão (nem precisam estar) ordenados para esta operação, e porque o requisito funcional exige uma pesquisa flexível — que encontre correspondências parciais e seja insensível a maiúsculas/minúsculas (“contém texto”), e não apenas correspondências exatas. A pesquisa binária não seria aplicável diretamente a este tipo de correspondência parcial sem pré-processamento adicional, pelo que a pesquisa linear é a escolha natural e mais simples para este requisito.

3.2.2. Pesquisa Binária — Pesquisa por Matrícula

Implementada na função `pesquisaBinariaMatricula`, este algoritmo exige que o array esteja ordenado pelo campo `matricula` — motivo pelo qual a função chama explicitamente `bubbleSortMatricula` antes de iniciar a pesquisa. O algoritmo mantém dois índices, `inicio` e `fim`, delimitando o intervalo de pesquisa, e em cada iteração calcula o elemento do meio ($\text{meio} = (\text{inicio} + \text{fim}) / 2$). Se o valor da matrícula no meio for igual ao procurado, o estudante é encontrado; se for menor, a pesquisa continua na metade direita ($\text{inicio} = \text{meio} + 1$); se for maior, continua na metade esquerda ($\text{fim} = \text{meio} - 1$). O processo repete-se até encontrar o elemento ou até o intervalo se esgotar ($\text{inicio} > \text{fim}$).

Complexidade: $O(\log n)$ para a pesquisa em si, após a ordenação prévia, que tem custo $O(n^2)$ com o Bubble Sort utilizado.

Justificativa da escolha: a pesquisa binária foi escolhida para a pesquisa por matrícula porque a matrícula é um valor numérico único, adequado a comparações de igualdade/ordem exatas, e porque permite demonstrar, na prática, a vantagem de uma estrutura ordenada: a complexidade logarítmica torna esta pesquisa significativamente mais eficiente do que a pesquisa linear para listas grandes. A reutilização da função de ordenação por matrícula (`bubbleSortMatricula`) já implementada para o requisito de ordenação reforça a coesão entre as duas funcionalidades do sistema (ordenar e pesquisar) e evita duplicação de código.

3.3. Quadro-Resumo Comparativo

Operação	Algoritmo	Critério / Campo	Complexidade	Pré-condição
Ordenação	Bubble Sort	Matrícula (crescente)	$O(n^2)$	Nenhuma
Ordenação	Selection Sort	Nome (alfabética)	$O(n^2)$	Nenhuma
Ordenação	Insertion Sort	Nota (decrescente)	$O(n^2)$	Nenhuma
Pesquisa	Pesquisa Linear	Nome (parcial)	$O(n)$	Nenhuma
Pesquisa	Pesquisa Binária	Matrícula (exata)	$O(\log n)$	Lista ordenada por matrícula

4. Manual de Utilização do Programa

Esta secção descreve, de forma prática, como compilar, executar e interagir com o sistema.

4.1. Requisitos do Sistema

- Sistema operativo Windows (devido ao uso das funções `system("cls")` e `system("dir /B *.txt > ...")`), ou um ambiente compatível (ex.: MinGW).
- Um compilador C instalado, por exemplo o GCC (recomenda-se o MinGW-w64 no Windows) ou o compilador incluído no Code::Blocks / Dev-C++.
- Um ficheiro de texto (.txt) com dados de estudantes, no formato Matrícula;Nome;Nota (um registo por linha), colocado na mesma pasta do executável (opcional, caso se pretenda carregar dados existentes).

4.2. Compilação do Programa

Assumindo que o código-fonte está guardado num ficheiro chamado, por exemplo, `sistema_alunos.c`, a compilação pode ser feita através da linha de comandos (terminal/Prompt de Comando), navegando até à pasta do ficheiro e executando:

```
gcc sistema_alunos.c -o sistema_alunos.exe
```

Este comando invoca o compilador GCC, compila o ficheiro de código-fonte e gera um executável chamado `sistema_alunos.exe`. Caso surjam avisos relacionados com funções como `scanf` ou `strcpy`, estes podem ser ignorados ou suprimidos com a flag `-Wno-deprecated-declarations`, não impedindo a compilação.

4.3. Execução do Programa

Após a compilação, o programa pode ser executado diretamente a partir da linha de comandos:

```
sistema_alunos.exe
```

Ao iniciar, o programa apresenta o menu principal com seis opções. A navegação é feita digitando o número da opção desejada e pressionando Enter.

4.4. Preparação dos Ficheiros de Dados

Para que a opção “1. Ler Dados dos Estudantes” funcione corretamente, deve existir pelo menos um ficheiro .txt na mesma pasta do executável, contendo registos no seguinte formato (um por linha, separados por ponto e vírgula):

1023;Joao Silva;15.50

1045;Maria Fernandes;18.00

1078;Pedro Costa;9.75

Cada linha deve conter exatamente três campos separados por “;”: a matrícula (numérica), o nome do estudante (apenas letras e espaços) e a nota (valor decimal). Linhas mal formatadas ou vazias são ignoradas pelo programa.

4.5. Navegação no Menu Principal

Opção	Descrição	Pré-condição
1	Lista os ficheiros .txt da pasta atual e permite escolher um para carregar os dados para a memória.	Deve existir pelo menos um ficheiro .txt na pasta.
2	Adiciona um novo estudante diretamente em memória, com validação de matrícula, nome e nota.	Nenhuma.
3	Acede ao submenu de ordenação e visualização da lista de estudantes.	Devem existir estudantes em memória (carregados ou adicionados).
4	Acede ao submenu de pesquisa de estudantes (por nome ou por matrícula).	Devem existir estudantes em memória.
5	Acede ao submenu de gravação dos dados em ficheiro (texto ou binário).	Devem existir estudantes em memória.
6	Termina o programa, libertando a memória alocada.	Nenhuma.

4.5.1. Submenu de Ordenação (Opção 3)

1. Opção 1: ordena a lista por matrícula, de forma crescente, e exibe a tabela resultante.

2. Opção 2: ordena a lista por nome, alfabeticamente, e exibe a tabela resultante.
3. Opção 3: ordena a lista por nota, de forma decrescente, e exibe a tabela resultante.
4. Opção 4: regressa ao menu principal.

4.5.2. Submenu de Pesquisa (Opção 4)

5. Opção 1: solicita um termo de pesquisa e apresenta todos os estudantes cujo nome contenha esse termo (pesquisa parcial, insensível a maiúsculas/minúsculas).
6. Opção 2: ordena automaticamente a lista por matrícula e solicita o número de matrícula a pesquisar, apresentando o estudante correspondente, caso exista.
7. Opção 3: regressa ao menu principal.

4.5.3. Submenu de Gravação (Opção 5)

8. Opção 1: grava os dados em memória num novo ficheiro de texto, com o nome “<NomeOriginal>_Saida.txt”, no formato Matrícula;Nome;Nota.
9. Opção 2: grava os dados em memória num ficheiro binário “<NomeOriginal>_Saida.dat”, contendo a representação binária das estruturas Estudante.
10. Opção 3: regressa ao menu principal.

4.6. Adicionar um Novo Estudante — Passo a Passo

11. No menu principal, escolher a opção 2.
12. Introduzir o número de matrícula (apenas dígitos). Se a entrada for inválida, vazia, com letras, ou se a matrícula já existir, o sistema gera automaticamente uma nova matrícula aleatória de 4 dígitos.
13. Introduzir o nome completo do estudante (apenas letras e espaços; números e caracteres especiais não são aceites e o sistema pedirá novamente).

14. Introduzir a nota, um valor numérico entre 0 e 20 (inclusive). Valores fora deste intervalo são rejeitados e o sistema pedirá novamente.
15. O sistema confirma a inserção com sucesso e o estudante passa a constar da lista em memória.

4.7. Dicas e Avisos Importantes

- Os dados adicionados ou ordenados em memória só são guardados de forma permanente após se utilizar a opção 5 (Gravar Dados em Ficheiro).
- Ao escolher carregar um novo ficheiro (opção 1), os dados anteriormente em memória são substituídos (a memória anterior é libertada antes de carregar o novo ficheiro).
- O programa foi desenvolvido para o ambiente Windows; em sistemas Linux/macOS, as chamadas `system("cls")` e `system("dir /B ...")` teriam de ser substituídas, respetivamente, por `system("clear")` e por uma listagem equivalente com `ls`.

5. Conclusão: Aprendizados e Desafios Enfrentados

5.1. Principais Aprendizados

O desenvolvimento deste sistema permitiu consolidar, de forma prática e integrada, vários conceitos centrais da programação em linguagem C. Em primeiro lugar, a gestão dinâmica de memória através de `malloc`, `realloc` e `free` revelou-se essencial para permitir que a lista de estudantes cresça de forma flexível, sem impor um limite fixo de registos definido em tempo de compilação.

Em segundo lugar, a implementação manual dos algoritmos de ordenação (Bubble Sort, Selection Sort e Insertion Sort) e de pesquisa (linear e binária) permitiu compreender, na prática, as diferenças de desempenho, complexidade e adequação de cada algoritmo a diferentes tipos de critérios (numéricos versus textuais, pesquisa exata versus pesquisa parcial), bem como a importância de pré-condições como a ordenação prévia exigida pela pesquisa binária.

Em terceiro lugar, a manipulação de ficheiros de texto e binários (com `fopen`, `fgets`, `strtok`, `fprintf` e `fwrite`) demonstrou como dados estruturados podem ser persistidos e recuperados entre execuções do programa, e evidenciou as diferenças práticas entre formatos legíveis por humanos (texto) e formatos compactos dependentes da representação interna das estruturas (binário).

Por fim, a construção de uma interface de menus robusta, com validação sistemática de entradas (verificação do retorno de `scanf`, validação de nomes apenas com letras, validação de notas dentro de um intervalo, geração automática de matrículas únicas) reforçou a importância de programar defensivamente, antecipando entradas inválidas por parte do utilizador.

5.2. Principais Desafios Enfrentados

- Gestão correta da memória dinâmica: garantir que o array é corretamente redimensionado com `realloc` a cada novo registo, e que a memória anteriormente alocada é libertada antes de carregar um novo ficheiro, evitando fugas de memória (memory leaks).
- Validação robusta de entradas mistas: lidar com a leitura intercalada de `scanf` (para números) e `fgets` (para texto), que exige cuidados especiais com o buffer de entrada para evitar comportamentos inesperados, como leituras vazias ou caracteres residuais.

- Garantir a pré-condição da pesquisa binária: assegurar que a lista está sempre ordenada por matrícula antes de aplicar o algoritmo de pesquisa binária, o que levou à decisão de invocar `bubbleSortMatricula` automaticamente dentro da função de pesquisa.
- Dependência de funcionalidades específicas do sistema operativo: o uso de `system("cls")` e `system("dir /B *.txt > ...")` para limpar o ecrã e listar ficheiros tornou o programa funcional e interativo, mas à custa da portabilidade, sendo um compromisso identificado como ponto de melhoria futura.
- Tratamento de casos-limite: assegurar que operações como ordenar, pesquisar ou gravar não são executadas quando a lista de estudantes está vazia, evitando comportamentos indefinidos sobre um array nulo.

5.3. Possíveis Melhorias Futuras

- Substituir as chamadas a `system("cls")` e `system("dir")` por alternativas portáteis (por exemplo, bibliotecas multiplataforma ou funções específicas para cada sistema operativo, selecionadas em tempo de compilação).
- Adicionar uma opção de ordenação otimizada (por exemplo, com paragem antecipada no Bubble Sort quando não ocorrem trocas numa passagem).
- Implementar um limite dinâmico (ou eliminar o limite fixo de 50 ficheiros) na listagem de ficheiros `.txt` disponíveis.
- Permitir a edição e remoção de registos existentes, complementando as operações de leitura, adição, ordenação, pesquisa e gravação já implementadas.

5.4. Considerações Finais

Em síntese, o sistema desenvolvido cumpre os objetivos propostos, constituindo uma aplicação funcional, modular e didaticamente rica, que integra de forma coerente estruturas de dados, algoritmos clássicos de ordenação e pesquisa, manipulação de ficheiros e validação de entradas. Os desafios encontrados ao longo do desenvolvimento contribuíram significativamente para a compreensão prática de conceitos fundamentais da linguagem C, e as limitações identificadas —

sobretudo a dependência de comandos específicos do Windows — representam oportunidades claras de evolução futura do projeto.

INTEGRANTES DO GRUPO

Nº	NOME COMPLETO	COTAÇÃO
1	ALFREDO DIAS BAMBA	
2	ALVES ADORADO	
3	FERNANDO DIASIVI SEBASTIÃO	
4	GABRIEL BULENVO MUANZA	
5	LIDIA FRANCISCO MONTEIRO	
XX	COTAÇÃO GERAL	